Project Proposal

End-to-End Data Engineering Pipeline for Brazilian E-Commerce Retail Analytics

Data Engineering Course | May 2026

1. Project Title

End-to-End Data Engineering Pipeline for Brazilian E-Commerce Retail Analytics

2. Team Members

Name	Role
Renato Silva	ETL Pipeline Engineer
Sandra Raj	Database Modeler & QA
Sudarsh Mekkampurath	Orchestration & Visualization

3. Dataset Summary

Source: Olist Brazilian E-Commerce Public Dataset (Kaggle)

URL: kaggle.com/datasets/olistbr/brazilian-ecommerce

Volume: ~100,000 orders across 9 interrelated CSV files, covering 2016 to 2018.

The dataset covers real orders from a Brazilian marketplace: orders, customers, products, sellers, payments, reviews, and geolocation in separate files. It covers project objectives due to:

- Nine relational tables require actual schema design and join logic, not just loading a flat file.
- 100k records is large enough to surface real data quality issues such as nulls, duplicates, and encoding problems.
- The data is static and public. Anyone on the team can pull the exact same files and reproduce the results.

4. Engineering Problem

The raw Olist data is nine separate CSV files with no enforced relationships between them. On its own, it cannot answer a single business question, or calculate delivery times without joining orders to products, or analyse revenue by region without linking customers to geolocation.

The engineering challenge is building a pipeline that cleans those files, resolves referential inconsistencies, and loads the data into a structured database where relationships are enforced and queries are reliable. That means ACID-compliant transactions, missing value handling, format standardization, and automated execution rather than a manual one-off script.

5. Project Goals

- Build a fully automated ETL pipeline that ingests, cleans, and loads all 9 Olist CSV files into PostgreSQL.
- Design a normalized relational schema (3NF) with proper primary and foreign key constraints.
- Apply transformation logic to resolve nulls, duplicates, type mismatches, and encoding issues before data reaches the database.
- Implement ACID-compliant transactions and data validation checks at load time.
- Orchestrate the pipeline with Apache Airflow using scheduled DAG execution.
- Connect Power BI directly to PostgreSQL and deliver a retail analytics dashboard covering delivery performance, revenue trends, seller rankings, and customer review scores.
- Produce complete documentation including an ER diagram, architecture diagram, and reproducible codebase on GitHub.

6. Proposed Solution / System

The proposed system follows a three-layer data engineering architecture that transforms nine raw CSV files into a structured, query-ready analytical database with a connected BI dashboard.

6.1 Architecture Overview

Raw CSV Files → Python ETL → Staging (Cleaned DataFrames) → PostgreSQL Data Warehouse → Power BI Dashboard

6.2 Layer 1: Extraction

All nine Olist CSV files are ingested using Python and Pandas. Each file is read into a separate DataFrame. File encoding issues (common in Brazilian Portuguese datasets) are handled at read time using UTF-8 with Latin-1 fallback. Basic schema validation is performed immediately after ingestion to confirm expected column names and row counts.

6.3 Layer 2: Transformation (Staging)

The following transformations are applied before loading into the warehouse:

- Null handling: rows with missing order IDs or customer IDs are flagged and isolated into a rejection log rather than silently dropped.
- Duplicate removal: exact duplicate rows are dropped; logical duplicates (same order_id with conflicting values) are escalated to a data quality report.
- Type standardisation: all date columns are parsed to datetime; zip codes are cast to strings to preserve leading zeros; numeric prices and freight values are cast to float.
- Referential integrity: orphaned foreign keys (e.g., order items referencing non-existent orders) are identified and logged before the load step.
- Derived columns: delivery_days (delivered_at − estimated_delivery), is_late (boolean), revenue_per_order (price + freight_value), and review_sentiment_label (mapped from review_score) are computed at staging.

6.4 Layer 3: Load — Relational Schema (3NF)

Cleaned data is loaded into PostgreSQL using SQLAlchemy with ACID-compliant transactions. The schema is normalised to Third Normal Form (3NF) and includes the following core tables:

Table	Key Columns & Relationships
Orders	PK: order_id | FK: customer_id → customers
customers	PK: customer_id | customer_unique_id, zip_code, city, state
order_items	FK: order_id → orders | FK: product_id → products | FK: seller_id → sellers
products	PK: product_id | category_name, weight, dimensions
sellers	PK: seller_id | zip_code, city, state
payments	FK: order_id → orders | payment_type, installments, payment_value
reviews	FK: order_id → orders | review_score, review_comment_title, review_comment_message
geolocation	PK: zip_code_prefix | latitude, longitude, city, state

6.5 Orchestration & Automation

The full pipeline is orchestrated using Apache Airflow. A single DAG (directed acyclic graph) defines the task sequence: ingest → validate → transform → load → quality-check → notify. Each task is independently retriable. The DAG is scheduled to run on-demand for this static dataset but is designed for periodic execution to support future live data integration. Power BI connects directly to PostgreSQL via a live connection for dashboard refresh.

7. Tools & Technologies

Category	Tool / Technology
Language	Python 3
Data Processing	Pandas, NumPy
Database	PostgreSQL
DB Connection	SQLAlchemy, psycopg2
Orchestration	Apache Airflow
Visualization	Power BI (connected to PostgreSQL)
Development	Jupyter Notebooks, VS Code
Version Control	Git & GitHub
Data Format	CSV (raw) -> PostgreSQL (processed)

8. Expected Deliverables

Deliverable	Description
etl.py	Python script for extraction, transformation, and loading
db_schema.sql	PostgreSQL schema with tables, keys, and constraints
main_pipeline.ipynb	Jupyter notebook documenting the full pipeline walkthrough
Airflow DAGs	Scheduled workflow definitions for automated pipeline execution
ER Diagram	Entity-relationship model of the PostgreSQL schema
Architecture Diagram	End-to-end flow: CSV -> Python -> PostgreSQL -> Power BI
Power BI Dashboard	Retail analytics dashboard connected to PostgreSQL
README.md	Setup instructions, tool list, and usage guide
Final Report	Completed 8-section project report per the course template

9. Timeline

Week	Milestone
Week 1	Dataset exploration, schema design, PostgreSQL setup, GitHub repo initialized
Week 2	ETL script development — extraction and cleaning (nulls, duplicates, type fixes)
Week 3	Database loading, schema validation, ACID compliance and constraint testing
Week 4	Airflow DAG setup, pipeline scheduling, data quality and lineage checks
Week 5	Power BI dashboard — connect to PostgreSQL, build retail KPIs and visuals
Week 6	Final report, architecture diagrams, documentation, submission packaging

10. Supervisor Feedback

Comments: